Computational Physics

Lecture 7: Non-linear equations and root-finding

Roots of non-linear equations

- Relaxation method
- Bisection method
- Newton's method
- Secant method

Maxima and minima of functions

- Golden ratio search
- Gradient descent

References: Chapter 6 of *Computational Physics* by Mark Newman

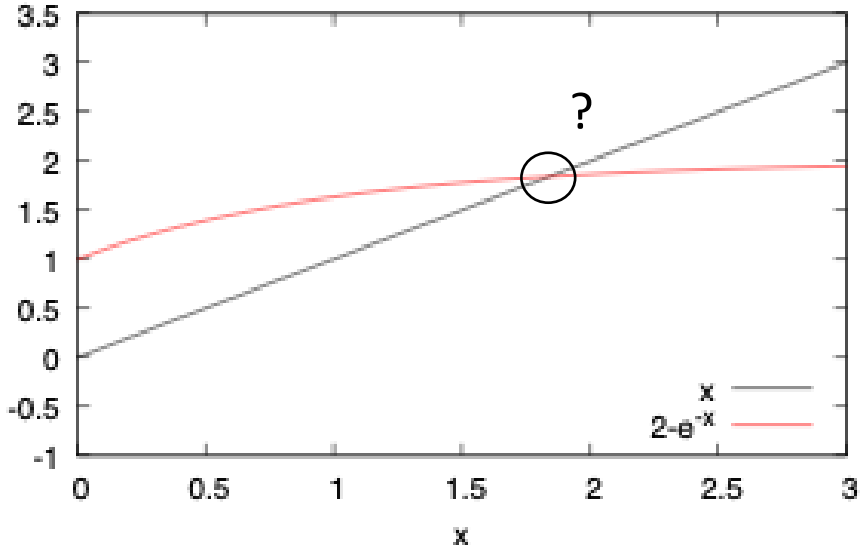
Chapter 9 of *Numerical Recipes Third Edition* by W.H. Press et al.

Non-linear equations

Suppose we have equation e.g. of the form:

$$x = f(x) \text{ or } f(x)=0$$

We can evaluate *f(x)*, but we do not know how to to know when *f(x)* takes respectively the value *x* or *0*



Examples:

- Roots of high-order polynomials (physics example: Lagrange L_1 point)
- Transcendental equations
  - e.g. magnetization equation $M = \mu \tanh \frac{JM}{k_B T}$

Relaxation method

Let's start for example with an equation with unknown solution

$$x = 2 - e^{-x}$$

The **relaxation method** involves guessing the solution, e.g. *x=1*, and plug it on the right side

$$x' = 2 - e^{-1} \simeq 1.632$$

The new value of x is not the guessed one, so we continue plugging it back

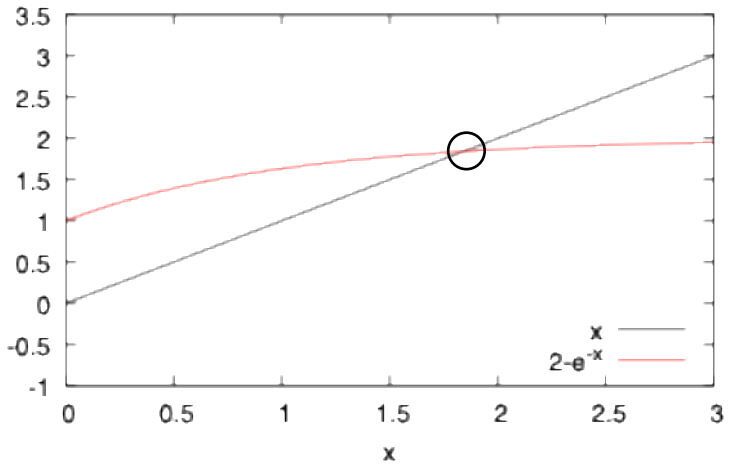
$$x'' = 2 - e^{-1.632} \simeq 1.804$$

Relaxation method

Doing this iteratively, and with some luck, our guess converge to a fixed point, i.e. it stops changing

```
import numpy as np
x = 1.0
for k in range(10):
    x = 2 - np.exp(-x)
    print(x)
```

```
1.6321205588285577
1.8044854658474119
1.8354408939220457
1.8404568553435368
1.841255113911434
1.8413817828128696
1.8414018735357267
1.8414050598547234
1.8414055651879888
1.8414056453310121
```



after 10 iterations it converges to *x=1.8414*. The more we continue the higher the digits precision

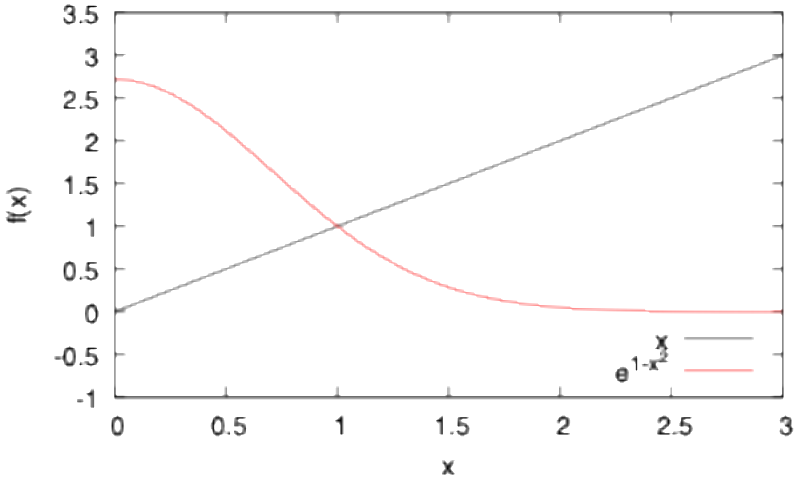
The improvement should be exponential

Problems:

- the equation needs to be expressed in the form x = *f(x)*
- what about multiple solutions? The convergence depends on the starting point. Best to have an idea of the starting point and start method with a guess near this one
- it might not converge at all

Relaxation method

Take e.g. $\log x + x^2 - 1 = 0$ can be re-expressed as $x = e^{1-x^2}$



We can see the solution is x=1, but we do not converge to it, but instead oscillates between two values:

```
import numpy as np
x = 0.5
for k in range(10):
    x =  np.exp(1-x**2)
    print(x)
```

```
2.117000016612675
0.030755419069985038
2.715711832754083
0.0017034651847384463
2.71827394057758
0.001679913095081425
2.7182741571849562
0.0016799111168229455
2.7182741572030236
0.0016799111166579386
```

If we use instead the alternative form $x = \sqrt{1 - \log x}$ it converges

```
import numpy as np
x = 0.5
for k in range(10):
    x =  np.sqrt(1-np.log(x))
    print(x)
```

```
1.3012098910475378
0.8583154914892762
1.0736775779454883
0.9637999044091371
1.0182689104343374
0.990906635925747
1.004557096969838
0.997724037576543
1.0011386299421705
0.9994308469350205
```

Relaxation method

Let's see now **how it mathematically works** and why it doesn't in some instances.

Suppose we have some *x=f(x)* for *x=x**, and that we start from a *x* close to *x**

Perform a Taylor expansion around x*. The value x' we will obtain after the iteration is

$$x' = f(x) = f(x^*) + (x - x^*) * f(x^*) + \ldots$$

$= x^*$

Thus we obtain the distance to the true solution without higher-order terms

$$x' - x^* = (x - x^*) f(x^*)$$

At each iteration the distance from the solution is multiplied by a factor f'(x*):

it works only if $|f'(x*)| < 1$

Relaxation method

This is the case of $x = e^{1-x^2}$ for which $\left|f'(x^*)\right| = \left|\left[-2x\, e^{1-x^2}\right]_{x=1}\right| = 2.$

Note that if |f'(x∗)| > 1 one can take $x = f(x)$ and invert it $x = f^{-1}(x)$ and it will converge

In fact take $x = f(u)$ and $u = f^{-1}(x)$. If we derive them

$$\frac{\mathrm{d}x}{\mathrm{d}u} = f'(u). \qquad \text{and} \qquad \frac{\mathrm{d}u}{\mathrm{d}x} = \frac{1}{f'(u)}$$

when $x = x^*$we have $u = f^{-1}(x^*) = x^*$ and so $\dfrac{\mathrm{d}u}{\mathrm{d}x} = \dfrac{1}{f'(x^*)}$

so the derivative of f^{-1} at x^* is simply the reciprocal of the derivative of f. If one is >1 the other is <1

Relaxation method: error

The quickest way to set accuracy is to observe two successive values and see when they stop changing up to a certain decimal. But we can obtain a more precise formula can be obtained in the following way:

Suppose you have the estimate x, so that the error δ with the true solution x* is $x^* = x + \delta$

At the next step, the error δ□ will be $x^* = x' + \delta'$

Subtracting them $x' - x = \delta - \delta'$

We also know $x' - x = (x - x^*)f'(x^*) \rightarrow \delta' = \delta\, f'(x^*)$

$$x' - x = \delta' \left[\frac{1}{f'(x^*)} - 1\right]$$

$$\delta' = \frac{x' - x}{1/f'(x^*) - 1} \approx \frac{x' - x}{1/f'(x) - 1}$$

error estimate on the nest step

Relaxation method: example

In the mean field theory of magnetism, we find a formula that relates the dependence of the magnetization M of a ferromagnetic material like iron on the temperature according to

$$M = \mu \tanh \frac{JM}{k_B T}$$

μ magnetic moment
J coupling constant
k_B Boltzmann's constant

Define m=M/μ and C= μJ/k_B $$m = \tanh \frac{Cm}{T}.$$

The equation surely admits m=0 as a solution, but varying T we obtain other solutions as well.

Assume C=1 and look for positive solutions at varying $T = [0.01,2]$ in intervals of 0.25.
Start from an initial guess of M=1 and find the solution up to a certain precision (e.g. 10^{-6}). You can also estimate the error directly as

$$\delta' = \frac{m' - m}{T \cosh^2(m/T) - 1}.$$

Relaxation method: example

```python
import numpy as np
import matplotlib.pyplot as plt

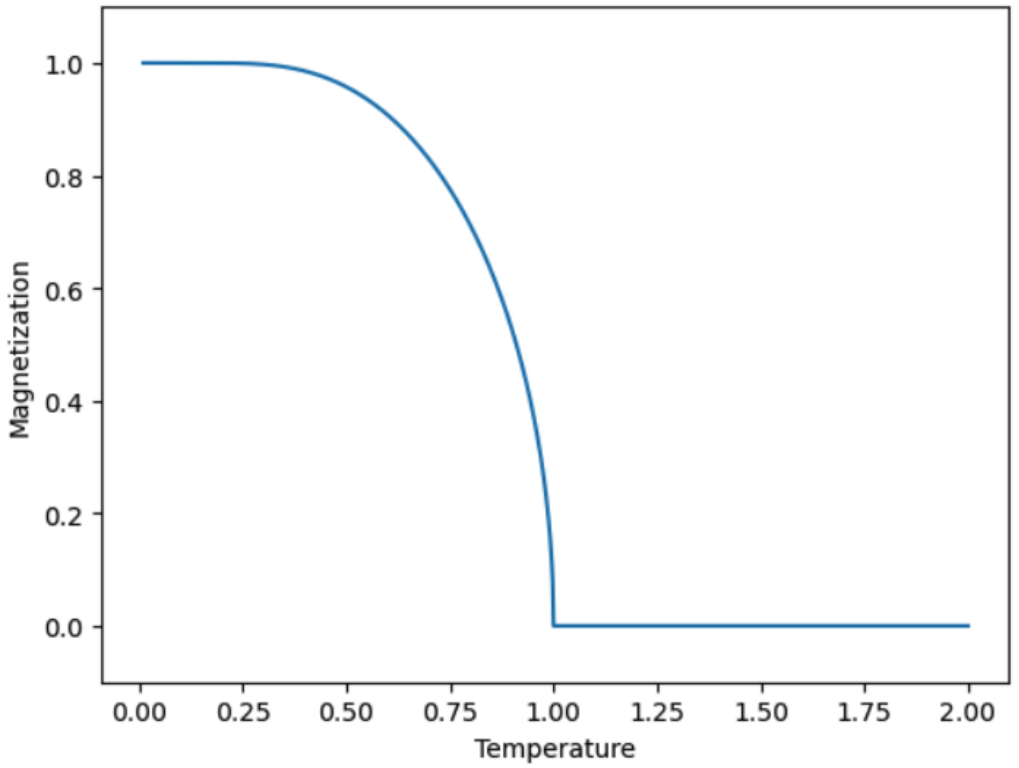
# Constants
Tmax = 2.0
points = 1000
accuracy = 1e-6

# Set up lists for plotting
y = []
temp = np.linspace(0.01,Tmax,points)

# Temperature loop
for T in temp:
    m1 = 1.0
    error = 1.0

    # Loop until error is small enough
    while error>accuracy:
        m1,m2 = np.tanh(m1/T),m1
        error = abs((m2-m1)/(T*np.cosh(m2/T)**2-1))
    y.append(m1)

# Make the graph
plt.plot(temp,y)
plt.ylim(-0.1,1.1)
plt.xlabel("Temperature")
plt.ylabel("Magnetization")
plt.show()
```



The magnetization is non-zero for T<1: if you heat a piece of magnetized iron the magnetization decreases and then drops to zero (phase transition). T=1 is the critical temperature

Relaxation method: two or more variables

Suppose you have simultaneous equations in multiple variables that can be expressed as

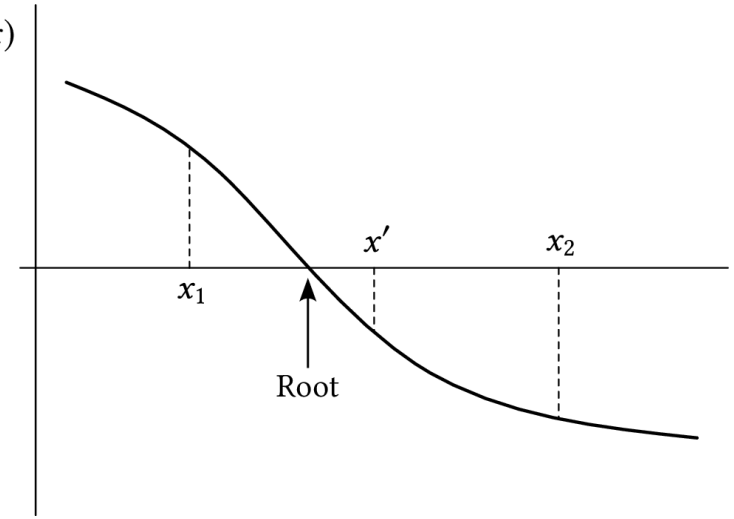
$$x_1 = f_1(x_1, \ldots, x_N)$$
$$\vdots$$
$$x_N = f_N(x_1, \ldots, x_N)$$

you can apply the same relaxation method: guess an initial condition and iterate until values do not change anymore.

It is not guaranteed to converge

Bisection (or binary) method

Another robust alternative method is **Bisection method or Binary search**.



Start from equation $f(x) = 0$

It means finding the zeros (or roots) of *f,* and any equation can be expressed as such)

Need to **specify an interval** $[x_1, x_2]$ where to search a solution.

Calculate $f(x_1)$ and $f(x_2)$. If there is a zero in between, one should be positive and the other negative (e.g. $f(x_1) > 0$ and $f(x_2) < 0$), and $f(x)$ must cross in between. And vice versa.

Take a new point in the middle of the interval $x' = \frac{1}{2}(x_1 + x_2)$

Check if $f(x') = 0$. Otherwise check if positive or negative. If e.g. $f(x') < 0$ it means root is in between $[x_1, x']$

Take a new point in the middle of the new interval $x' = \frac{1}{2}(x_1 + x')$

etc.

Bisection (or binary) method

Repeat the process until distance between two points is within accuracy
If the root exist, the method will find it

Still exponentially fast: if $x_2 - x_1 = \Delta$ and we halve at each step the distance, we get the distance between points after N steps to be $\Delta/2^N$

When accuracy ε is reached $\varepsilon = \Delta/2^N$ or $N = \log_2 \frac{\Delta}{\varepsilon}$
even if initial $\Delta \gg \varepsilon$ the value of N remains contained. So no need to worry about size of interval $[x_1, x_2]$

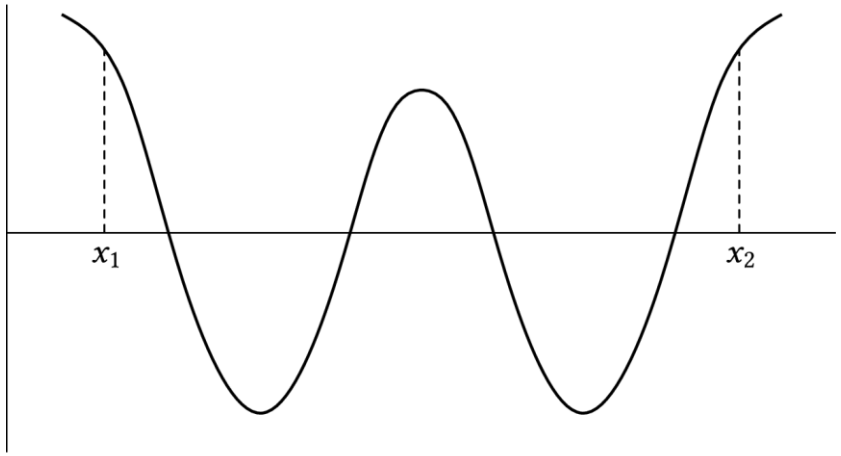
Bisection (or binary) method

Problems:

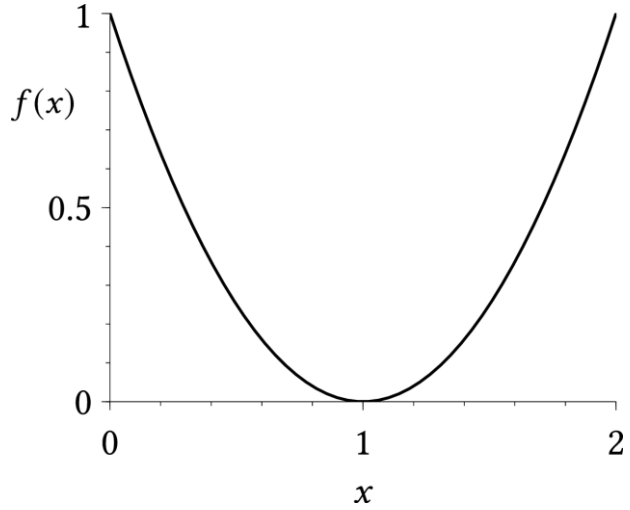
1) if $f(x_1)$ and $f(x_2)$ have same sign, can mean:

- no root
- even number of roots (not easy to make it work)

maybe increase interval $[x_1, x_2]$ and check again



2) Good luck finding a root starting from any x_1, x_2 choice of points. Better to have some knowledge to help guess correctly



3) Cannot find roots in even-order polynomials, i.e. double or quadruple roots that fall in the same place, e.g. $(1-x)^2$
The function for these roots do not cross the x-axis

4) Not extendable to multiple equations

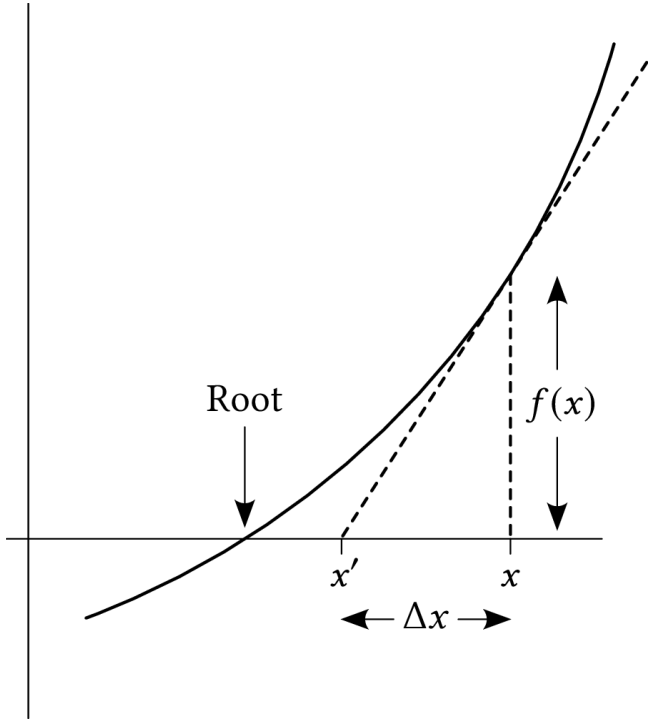
Newton's (or Newtons-Raphoson) method

Start from equation $f(x) = 0$

Guess a value x where root is located (should be close)

Use the slope $f'(x)$ to extrapolate where root is (dashed line)

$$f'(x) = \frac{f(x)}{\Delta x}$$



Obtain the new estimate

$$x' = x - \Delta x = x - \frac{f(x)}{f'(x)}$$

Iterate up to convergence

Newton's method: error

To estimate error, perform Taylor expansion around root x^* (remember $f(x^*) = 0$)

$$0 = f(x^*) = f(x) + (x^* - x)\, f'(x) + \frac{1}{2}(x^* - x)^2 f''(x) + \cdots$$

Rearrange separating x^*

$$x^* = \boxed{x - \frac{f(x)}{f'(x)}} - \frac{1}{2}(x^* - x)^2 \frac{f''(x)}{f'(x)} + \cdots \longrightarrow x^* = x' - \boxed{\frac{1}{2}(x^* - x)^2 \frac{f''(x)}{f'(x)}} + \cdots$$

estimate x' in Newton's method

error

How the error δ changes after two steps: $x^* - x = \delta$ $x^* - x' = \delta'$

$$\delta' = x^* - x' = -\frac{1}{2}(x^* - x)^2 \frac{f''(x)}{f'(x)} = -\frac{1}{2}\frac{f''(x)}{f'(x)}\delta^2$$

We have a **quadratic convergence** (quick convergence**)**: if error is δ at one step, it becomes δ^2 at the next one

In practice given $x^* = x + \delta = x' + \delta' = x + c\delta^2$, estimate error δ on *x* as difference between next x' value:

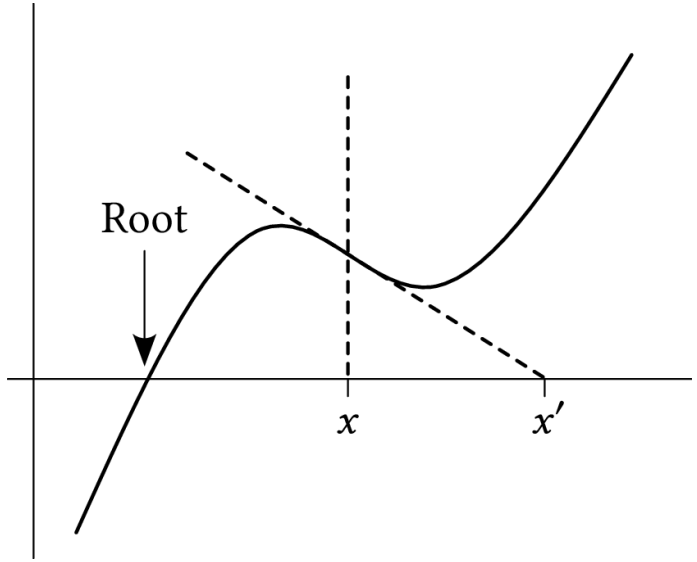
$x' - x = \delta - c\delta^2 \approx \delta$ so can stop when difference between x' and x is within accuracy

Newton's method

Disadvantages:

it requires the derivative $f'(x)$

does not always converge, e.g. if slope of *f(x)* is in the wrong direction



Newton's method: example

Compute inverse hyperbolic tangent value *x* for a given value *u* (which is an input constant)

$u = \tanh x$ looks like a problem $f(x) = 0$ with $f(x) = \tanh x - u$

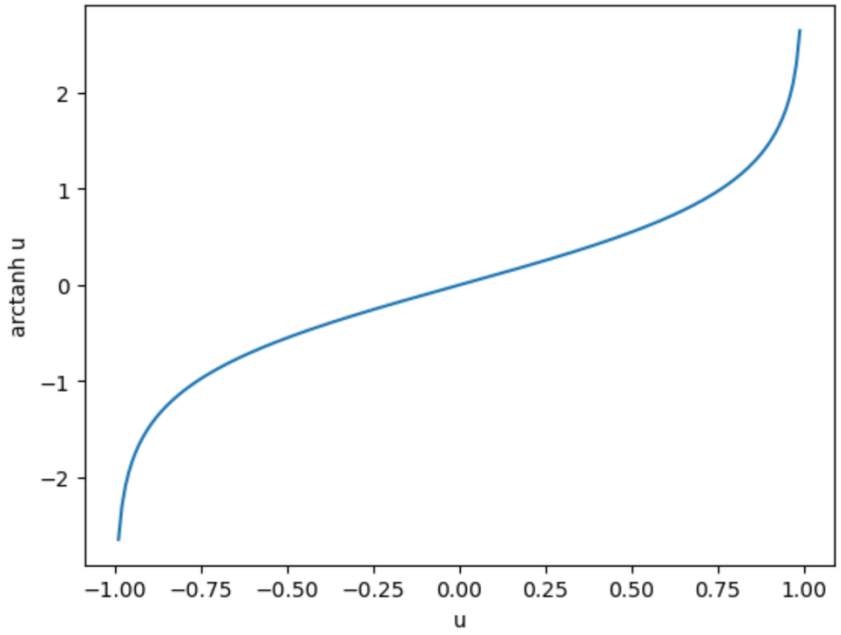
Given that $f'(x) = \frac{1}{\cosh^2 x}$ use Newton's method to compute the root x at a given u, and compute arctanh(*u*) with *u* in between [-0.99,0.99]

```
import numpy as np
import matplotlib.pyplot as plt

accuracy = 1e-12

def arctanh(u):
    x = 0.0
    delta = 1.0
    while abs(delta)>accuracy:
        delta = (np.tanh(x)-u)*np.cosh(x)**2
        x -= delta
    return x

upoints = np.linspace(-0.99,0.99,200)
xpoints = []
for u in upoints:
    xpoints.append(arctanh(u))
plt.plot(upoints,xpoints)
plt.show()
```

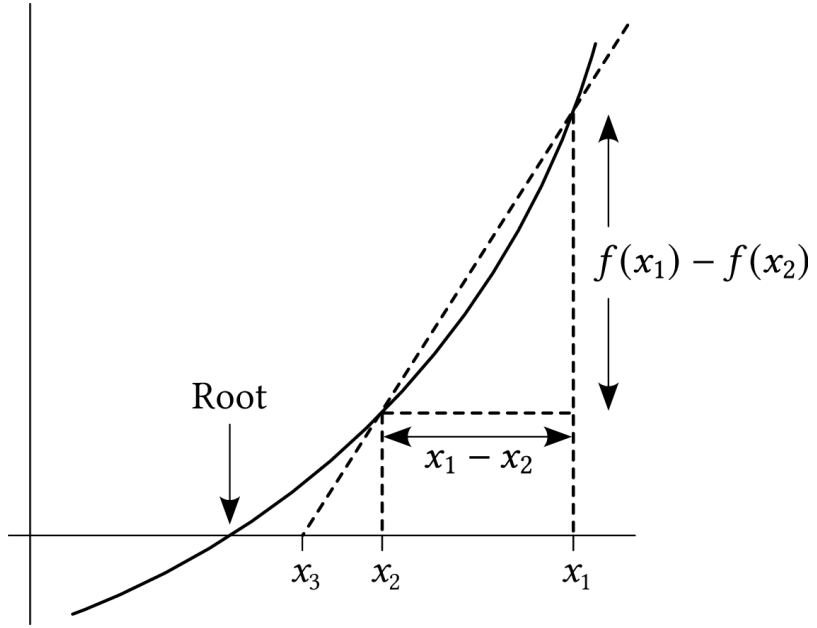


Secant method

When we do not know the derivative, we can use this variant of Newton's method

Start with two points x_1 x_2 (which do not necessarily bracket the root!) and compute the approximated derivative at x_2

$$f'(x_2) \simeq \frac{f(x_1) - f(x_2)}{x_1 - x_2}$$



The Newton's estimate of the root position x_3 becomes

$$x_3 = x_2 - \frac{f(x_2)}{f'(x_2)} \simeq x_2 - f(x_2)\frac{x_1 - x_2}{f(x_1) - f(x_2)}$$

Use then x_3 and last estimate x_2 to estimate new value for root

Newton's method: two or more variables

Suppose you have simultaneous equations in multiple variables that can be expressed as

$$x_1 = f_1(x_1, \ldots, x_N)$$
$$\vdots$$
$$x_N = f_N(x_1, \ldots, x_N)$$

Suppose you have a root at $x_1^*, \ldots, x_N^*$. If you write the Taylor expansion

Jacobian matrix $J_{ij} = \dfrac{\partial f_i}{\partial x_j}$

$$0 = f_1(x_1^*, \ldots, x_N^*) = f_1(x_1, \ldots, x_N) + \sum_j \left(x_j^* - x_j\right) \frac{\partial f_i}{\partial x_j} + \cdots \quad \text{or} \quad \mathbf{0} = \mathbf{f}\left(\mathbf{x}^*\right) = \mathbf{f}\left(\mathbf{x}\right) + \mathbf{J} \cdot \left(\mathbf{x}^* - \mathbf{x}\right) + \ldots$$

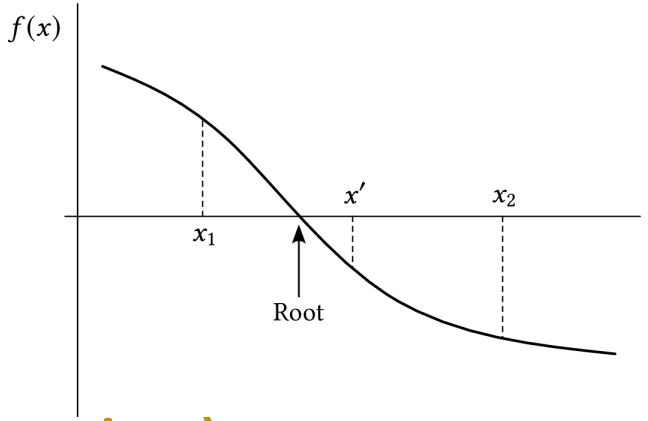
Using the same arguments we obtained the next estimate of $\mathbf{x}'$ via:

$$\mathbf{J} \cdot \left(\mathbf{x} - \mathbf{x}'\right) = \mathbf{f}\left(\mathbf{x}\right) \quad \text{or} \quad \boxed{\mathbf{J} \cdot \boldsymbol{\Delta}\mathbf{x} = \mathbf{f}\left(\mathbf{x}\right)}$$

System of linear equations to solve in $\boldsymbol{\Delta}\mathbf{x}$ (using e.g. LU decomposition) to obtain new root estimate $\mathbf{x}'$

Summary

Bracketing method



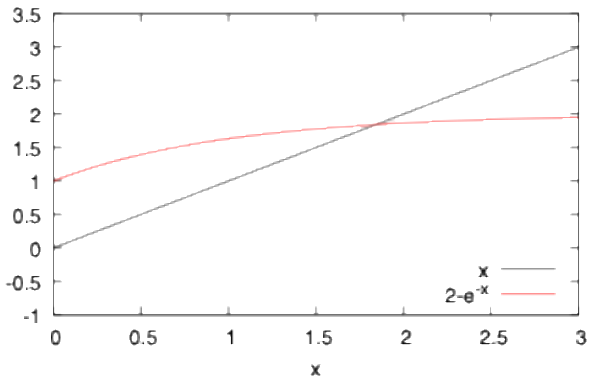
Bisection (or binary) method:

- Guaranteed to converge with a fixed rate
- Need to bracket the root

Local methods (all extendable in multi-dimensions)

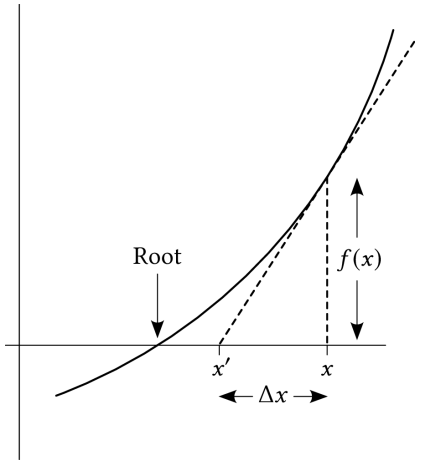
Relaxation method:

- Simple to implement
- Does not require derivative
- Need *x=f(x)*
- Often does not converge



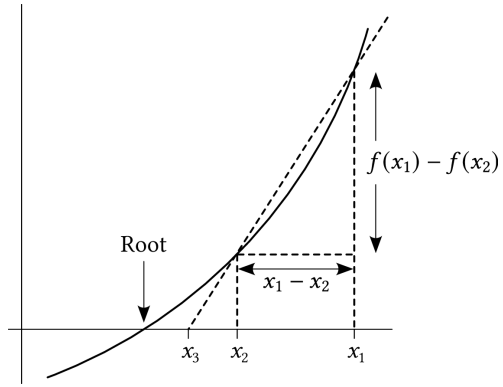
Newton method:

- Very fast when converges
- Can be sensitive to initial guess
- May not converge if $f'(x) = 0$
- Requires evaluation of the derivative at each step



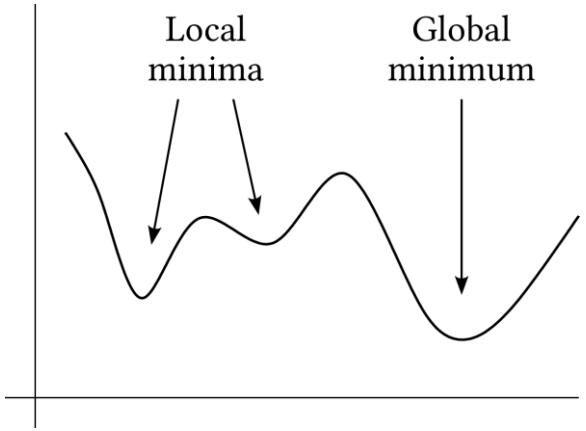
Secant method:

- Similar properties of Newton method
- Does not need derivative



Maxima and Minima of functions

Often, we are interested to **find the minimum/maximum of one single/multi variable function** (also known as **optimization problems**)



Problem connected to:
-energy minimization of potential energy
-variational method in quantum mechanics

Functions can have multiple local minima/maxima, and a global minimum/maximum

Local minima correspond to metastable states, and local maxima are the barriers that set the energy required to hop from one minimum to another.

A way to compute minima/maxima for a function $f(x_1, \ldots, x_N)$ is to calculate partial derivatives for each argument and solve this set of equations (we know how to solve them if linear or non-linear)

$$\frac{\partial f}{\partial x_i} = 0 \quad \forall i$$

Let's see other optimization methods: golden ratio search and gradient descent

Golden section search

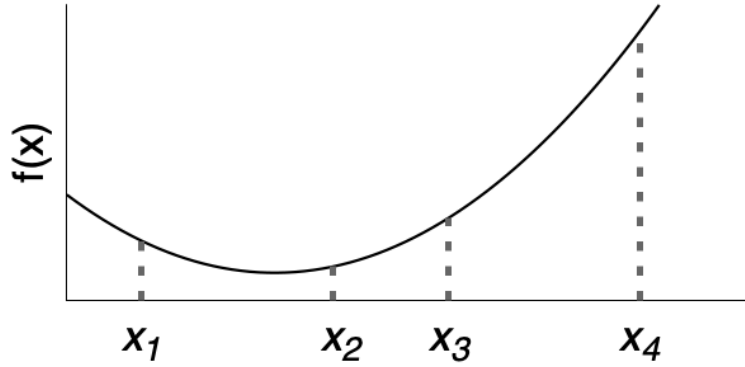
Suppose we want to evaluate a local minimum of f(x) (equivalent to finding maximum of –f(x)). The method does not tell if it also global minimum

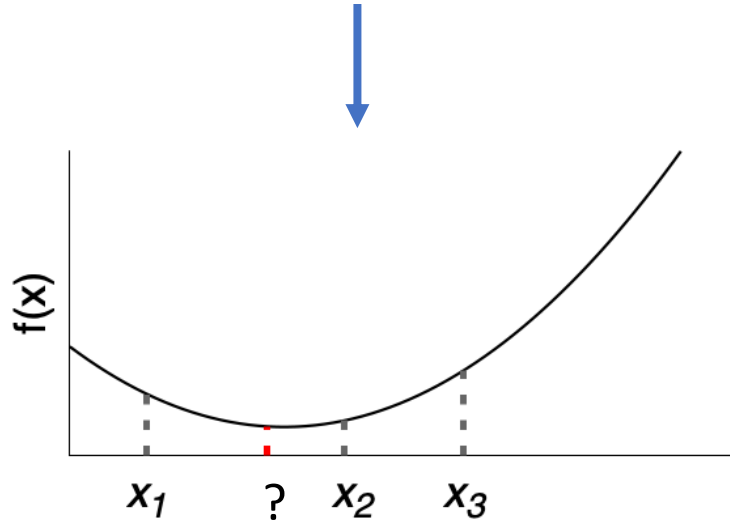
Start considering four non-evenly spaced points x_1, x_2, x_3, x_4

Suppose either $f(x_2)$ or $f(x_3)$ are smaller than $f(x_1)$ or $f(x_4)$: then a minimum should be present in $[x_1, x_4]$

If e.g. $f(x_2)$ is smallest, minimum should lie in $[x_1, x_3]$:
repeat iteration re-using x_1, x_2, x_3 as new points and add another intermediate point to perform iteration again

How to choose this new point?





Golden section search

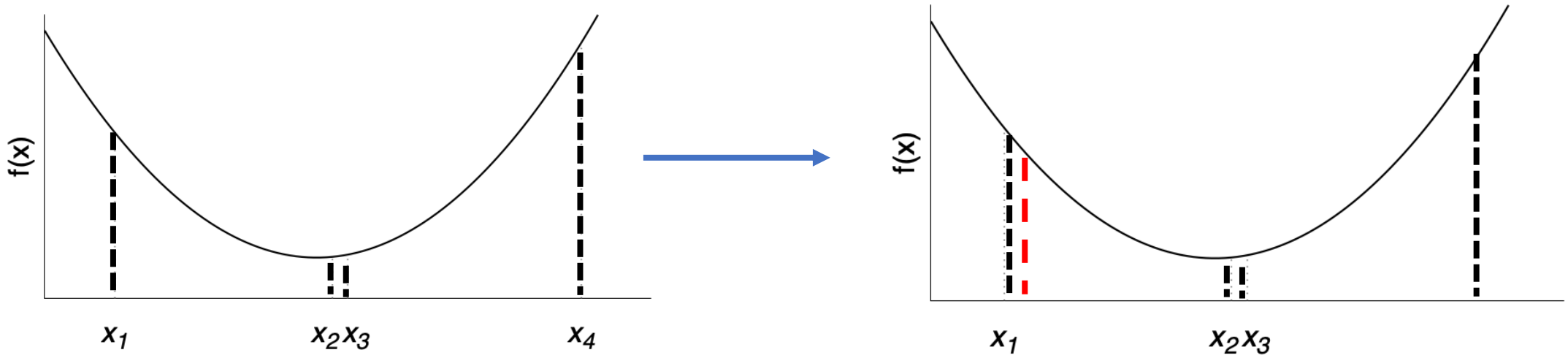
Let's think using the four reference points x_1, x_2, x_3, x_4. We need to maximize the interval reduction at each step

They should be symmetric around the midpoint of $[x_1, x_4]$ interval:

$$x_2 - x_1 = x_4 - x_3$$

but there is still the choice of the distance from the midpoint

If x_2 is too close to midpoint it maximize reduction of interval but in the next step x_1 and the new point (chosen) will be too close (taking the new point symmetrically to x_2)

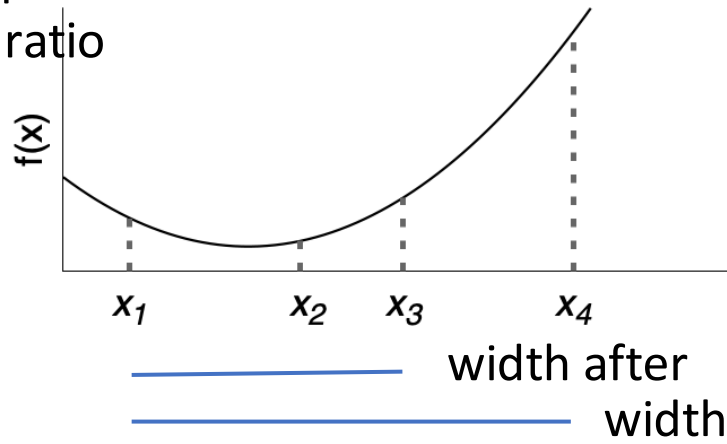


Golden section search

Instead choose position of x_2 such that it is equally closer to the center in both steps
The width of the bracket before is $x_4 - x_1$ and after will be $x_3 - x_1$.Consider their ratio

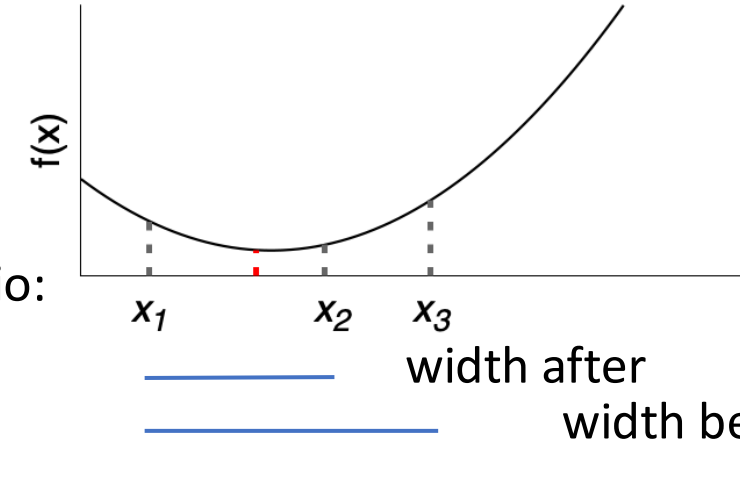
$$x_2 - x_1 = x_4 - x_3$$

$$z = \frac{x_4 - x_1}{x_3 - x_1} = \frac{x_2 - x_1 + x_3 - x_1}{x_3 - x_1} = \frac{x_2 - x_1}{x_3 - x_1} + 1$$



In the next step the bracket widths would be before $x_3 - x_1$ and after $x_2 - x_1$
with a ratio

$$z = \frac{x_3 - x_1}{x_2 - x_1}$$



If we want the ratio *z* to remain the same before and after, we get the goldent ratio:

$$z = \frac{1}{z} + 1 \longrightarrow z^2 - z - 1 = 0 \longrightarrow z = \frac{1 + \sqrt{5}}{2} = 1.618$$

So x_3 should be 1/z the way between x_1, x_4 and x_2 in the mirror position around the center point

Golden section search

Golden ratio search algorithm:

1. Choose x_1, x_4 and compute $\quad x_2 = x_4 - \frac{x_4 - x_1}{z} \qquad x_3 = x_1 + \frac{x_4 - x_1}{z}$

Evaluate also *f(x)* at each point and check that either $f(x_2)$ or $f(x_3)$ is smaller than $f(x_1)$ or $f(x_4)$

2. If $f(x_2) < f(x_3)$: the new interval is x_1, x_3
   x_3 becomes the new x_4
   x_2 becomes the new x_3 (only option based on *z* value)
   need to compute new x_2 computed using the same equation in 1)

   If $f(x_2) > f(x_3)$: the new interval is x_2, x_4
   x_2 becomes the new x_1
   x_3 becomes the new x_2
   need to compute new x_3 computed using the same equation in 1)

3. Evaluate *f(x)* at new point and repeat process from point 2).

At each iteration check if $x_4 - x_1$ is within target accuracy. In this case, stop process and the final estimate of the minimum minimum position will be $\frac{x_4 - x_1}{2}$

Golden section search

Observations:

The method needs an initial bracketing in which one of the two middle points is lower. this guarantees to find at least one minimum

the method converges exponentially like bisection method: the interval decreases by factor *z=1.618* at each iteration no matter what (for bisection is faster with a factor is 2)

No direct generalization for multiple variables

Gradient descent

A method that does not need bracketing and can be easily extended in multiple dimensions

Maxima and minima are just roots of $f'(x) = 0$. One can directly apply Newton's method, replacing $f(x)$ with $f'(x)$ in the formula

$$x' = x - \frac{f'(x)}{f''(x)}$$

Quadratic (fast) convergence. Approach widely used training large neural networks.

Gradient descent

If we don't know *f''(x)*, we can use approach of secant method.

Start from two points x_1, x_2 , estimate the second derivative in x_2 $\quad f''(x_2) \simeq \frac{f'(x_1) - f'(x_2)}{x_1 - x_2}$

use it to estimate third point x_3 $\quad \boxed{x_3 = x_2 - \frac{f'(x_2)}{f''(x_2)} \simeq x_2 - f'(x_2)\frac{x_1 - x_2}{f'(x_1) - f'(x_2)}}$

Alternatively, I can express it as

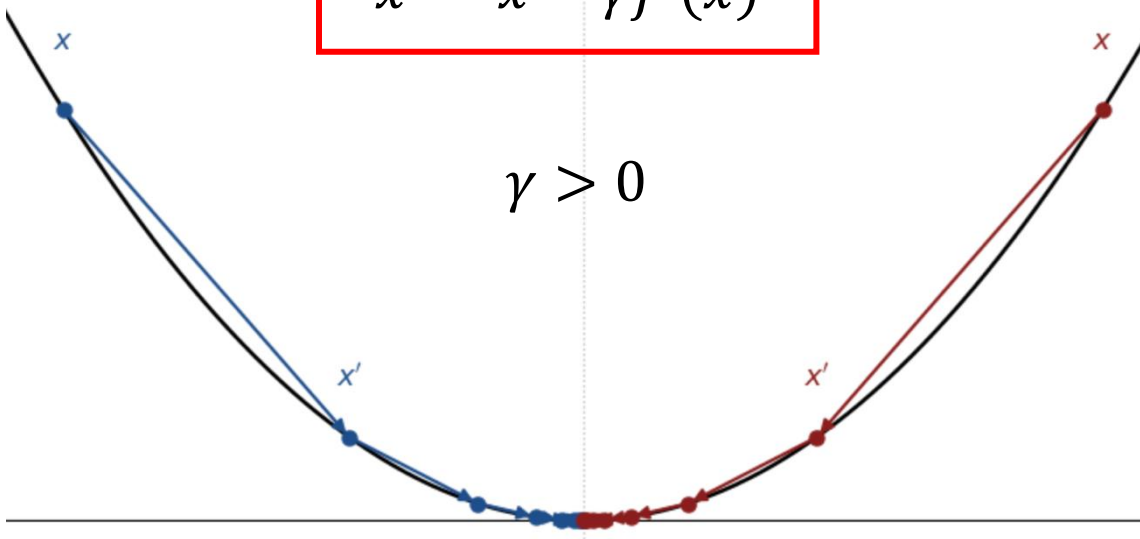
$$\boxed{x' = x - \gamma f'(x)}$$ **Method of gradient descent**

One does not need to know perfectly $1/f''(x)$, but can just input a constant γ which is a rough guess of $1/f''(x)$, it just needs to be of the same order of magnitude, and the algorithm will work

Gradient descent

We measure the gradient $f'(x)$ at x, and then subtract that gradient times a constant γ

$$x' = x - \gamma f'(x)$$



$\gamma > 0$ the point will always move downhill to the minimum

$\gamma < 0$ the point will always move uphill to the maximum

Magnitude of γ controls rate of convergence:

-too small and it takes time.

-too large and it is quicker, but too much and you can overshoot the minimum.

Best guess would be $1/f''(x)$, but we can do trial/error if we don't know

Gradient descent

If we don't even know the first derivative use $$f'(x_2) \simeq \frac{f(x_1) - f(x_2)}{x_1 - x_2}$$

$$x_3 = x_2 - \gamma \frac{f(x_1) - f(x_2)}{x_1 - x_2}$$

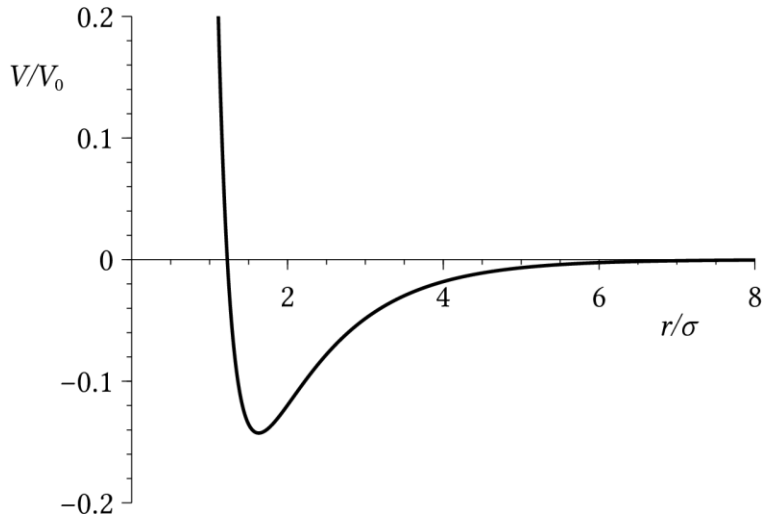
Easily generalizable to multivariable

$$\mathbf{x}' = \mathbf{x} - \gamma \nabla f(\mathbf{x})$$

Can be used to bring a particle's system initial conformation with overlaps to an energy minimum

Minimum and maximum: example

The Buckingham potential is used to approximate interactions between atoms in solids or gases



$$V(r) = V_0\left[\left(\frac{\sigma}{r}\right)^6 - e^{-r/\sigma}\right]$$

Combines two terms: a short-range repulsive part (core repulsion) and a longer-range attractive force (dipole interactions)

At intermediate distance two parts balance out, giving a minimum corresponding to resting distance between atoms

1) Write a golden ratio search program to find minimum
2) Write a program using the gradient descent. Use different γ and starting points to see the difference

Note that σ sets the scale for the curve. Use σ=1nm. No need to know V_0 value

As an extra, you can save values of x_1, x_2, x_3, x_4 at each step and plot them as a function of the iteration step, to see how the program evolves their position. Same for gradient descent, saving the position at each time.

Minimum and maximum: example with golden search

```python
import numpy as np

# Constants
sigma = 1.0             # Value of sigma in nm
accuracy = 1e-6         # Required accuracy in nm
z = (1+np.sqrt(5))/2       # Golden ratio

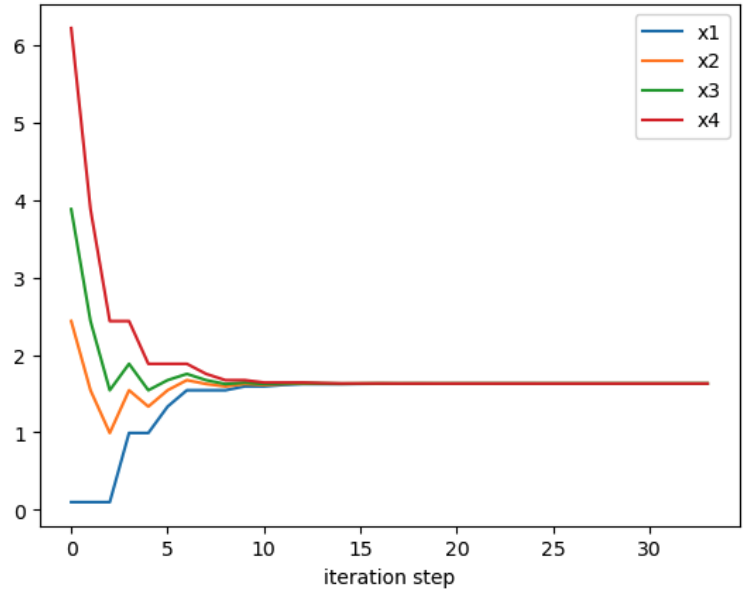
# Function to calculate the Buckingham potential
def f(r):
    return (sigma/r)**6 - np.exp(-r/sigma)

# Initial positions of the four points
x1 = sigma/10
x4 = sigma*10
x2 = x4 - (x4-x1)/z
x3 = x1 + (x4-x1)/z

# Initial values of the function at the four points
f1 = f(x1)
f2 = f(x2)
f3 = f(x3)
f4 = f(x4)

# Main loop of the search process
while x4-x1>accuracy:
    if f2<f3:
        x4,f4 = x3,f3
        x3,f3 = x2,f2
        x2 = x4 - (x4-x1)/z
        f2 = f(x2)
    else:
        x1,f1 = x2,f2
        x2,f2 = x3,f3
        x3 = x1 + (x4-x1)/z
        f3 = f(x3)

# Print the result
print("The minimum falls at",0.5*(x1+x4),"nm")
```



Minimum and maximum: example with gradient descent

```python
import numpy as np
import matplotlib.pyplot as plt

sigma = 1.0
accuracy = 1e-6
gamma = 1
nsteps = 1000

def f(r):
    return (sigma / r) ** 6 - np.exp(-r / sigma)
def fp(r):
    return -6.0 * sigma**6 / r**7 + np.exp(-r / sigma) / sigma

x = 5
fx = f(x)
listx = [x]
listf = [fx]

for step in range(0,nsteps):
    x_new = x - gamma * fp(x)
    listx.append(x_new)
    listf.append(f(x_new))
    if abs(x_new - x) < accuracy:
        x = x_new
        break
    x = x_new
```

Several things can go wrong:

- If γ is too small and you stop by checking the change in x, you may exit early because the steps become very slow.
- If γ is too large, the steps overshoot the minimum: the method oscillates or diverges.
- Stopping when f'(x) is very small can fail in a flat region (try using a large γ).
- A while loop with a very small γ may never finish, because the point barely moves.

